ARM Register Sets — Exam Ready Notes

Source: Module 5 “The ARM Instruction Set” pp.14–17. Highlighted italic lines are **comments**.

1. Headline facts (module pp.14–17)

- ARM has **37 registers, all 32 bits long**: **1** dedicated program counter + **1** CPSR + **5** dedicated saved program status registers + **30** general-purpose registers.
- The **current processor mode governs which bank is visible**: each mode sees its own r13 (sp) and r14 (lr) — FIQ also its own r8–r12 — while r0–r12, r15 (pc) and CPSR are common to all modes. Privileged modes (except System) additionally see their private **SPSR**.
- **System mode uses the User-mode register set** (module note, p.17).
- Visible at any moment: 16 data registers (r0–r15) + CPSR (+ SPSR in an exception mode). The register file has **3 read ports**, so most instructions fetch all operands in one cycle — the RISC “register-rich” idea.

2. The register banks (organisation summary)

Register	User / System	FIQ	IRQ	SVC	Abort	Undef
r0–r7	shared	shared	shared	shared	shared	shared
r8–r12	shared	**FIQ_r8–FIQ_r12**	shared	shared	shared	shared
r13 (sp)	SP_usr	**SP_fiq**	**SP_irq**	**SP_svc**	**SP_abt**	**SP_und**
r14 (lr)	LR_usr	**LR_fiq**	**LR_irq**	**LR_svc**	**LR_abt**	**LR_und**
r15 (pc)	shared	shared	shared	shared	shared	shared
CPSR	shared	shared	shared	shared	shared	shared
SPSR	—	**SPSR_fiq**	**SPSR_irq**	**SPSR_svc**	**SPSR_abt**	**SPSR_und**

Bold = private (banked) copy. Count check: 17 shared (r0–r15 + CPSR) + 5 (FIQ r8–r12) + 5×(sp+lr) + 5×spsr = 17+5+10+5 = 37 registers.

3. Special function registers

Register	Role (module p.16)
PC — r15	Program counter. Any instruction with **PC as its destination register is a program branch**.
LR — r14	Link register. Saves a copy of PC when executing **BL** (subroutine call) or on jumping to an exception/interrupt routine; copied back to PC on return.
SP — r13	Stack pointer. There is **no stack in the ARM architecture** — R13 is merely *reserved* as the pointer for a program-managed stack.
CPSR	Current Program Status Register — holds the visible status: flags **N Z C V**, interrupt disables I/F, T bit and **mode bits M[4:0]**.
SPSR	Saved Program Status Register — holds a copy of the previous CPSR while an exception/interrupt runs; copied back to CPSR on return. **Not available in User or System mode.**

4. Comments — the “why” that earns marks

*Comment 1: banking is a **performance** feature. On an exception the hardware hands the new mode its own sp/lr/spsr, so nothing must be pushed to memory first → minimal interrupt latency; the same trick makes mode switches cheap.*

*Comment 2: FIQ gets **seven** private registers (r8–r14) — the most of any mode — because the fast interrupt must spend (almost) zero time saving context. IRQ gets only two (sp, lr).*

Comment 3: “no hardware stack” is a classic trick line: the architecture defines no PUSH/POP hardware; SP is a convention, and even PUSH/POP are just LDM/STM using r13. The stack is entirely software-managed.

*Comment 4: link to pipelining — while an instruction executes, r15 is already two fetches ahead, which is why **PC reads as PC+8** in ARM state (PC+4 in Thumb).*

5. One-minute revision

Numbers: 37 registers / 32-bit · 30 GPRs · 5 SPSRs · FIQ banks r8–r14 · others bank r13/r14(+SPSR) · no SPSR in User or System · System = User’s register set.

Likely questions: (1) Describe the ARM register organisation with the banking table. (2) Explain PC, LR, SP, CPSR and SPSR. (3) Why does ARM bank registers? (4) How many registers are visible in User mode and why?

Comment: always draw the bank table (section 2) in the exam — it converts a prose answer into a diagram and scores full marks.